3D Indoor Space Sizing for Drone Flight Planning Using Stereoscopic Imagery

Tendai Chimuka*, Yihan Xu†, Ziqian Dong†, and Roberto Rojas-Cessa*
*New Jersey Institute of Technology, Newark, NJ 07102, USA
†New York Institute of Technology, New York, NY 10023, USA
Email: {ttc, rojas}@njit.edu, {yxu66, ziqian.dong}@nyit.edu

***Abstract*—Indoor uncrewed aerial vehicles (UAVs) require local perception of the surroundings where satellite signal for GPS is unavailable. Although LiDAR-based system is a popular alternative, it adds cost, power demand, and payload mass. Therefore, we resort to using stereoscopic images to detect obstacles and retrieve distance measurements from the images to determine hazards and 3D space available for flying the indoor drone. The experimental work is based on two Logitech C615 webcams, separated by 0.25 m, that capture 1280×720 image pairs. After calibration and rectification, dense disparity is converted into metric depth and a three-dimensional point cloud. The reconstruction estimates visible dimensions, obstacle locations, opening widths, and flight clearances. Experiments compare these estimates with physical measurements and examine the effects of range, lighting, surface texture, and stereo-matching quality. The study assesses whether consumer cameras can provide useful local spatial information for indoor UAV path planning.**

***Index Terms*—Disparity estimation, flight path planning, indoor navigation, point cloud, stereo vision, three-dimensional reconstruction, uncrewed autonomous vechicle**

I. Introduction

Indoor Uncrewed Autonomous Vehicles (UAVs) need on-the-fly perception to detect obstacles, judge clearances, and plan a safe path when satellite navigation is unavailable. Existing platforms pair Light Detection and Ranging (LiDAR) with inertial sensors, cameras, and a compute board. On a small airframe, the cost, weight, and power draw usually rule that combination out.

Stereo vision recovers depth from the disparity between two images taken at a given distance apart. Calibrated disparities become a point cloud of whatever surfaces both cameras can see. Accuracy decreases with range and degrades further under poor lighting, on weak or repetitive texture, and around reflections and occlusions [1].

This work tests whether two Logitech C615 webcams on a 0.25 m baseline can size indoor spaces from 1280×720 images. The pipeline runs ChArUco calibration, rectification, dense stereo matching, metric-depth conversion, and point-cloud reconstruction, and reports the visible size of a selected region.

We describe the pipeline and the conditions of each stage and test the reconstruction on a computer-generated scene of a given size. A real capture cannot separate algorithm error from calibration, rig, and scene error, because they arrive together in one measurement. Verifying the algorithm on ideal input, we first attribute the remaining error to capture conditions under user control.

II. Related Work

Autonomous flight without Global Positioning System (GPS) in obstacle cluttered environments has been extensively investigated. Mohta et al. [2] integrated sensing, state estimation, mapping, and control for fast quadrotor flight. Zhou et al. [3] generated trajectories for aggressive flight in constrained spaces, and Kong et al. [4] detected and avoided small dynamic obstacles onboard. These studies confirm the value of local three-dimensional perception using onboard sensing and computing.Open-source visual-inertial simultaneous localization and mapping (SLAM) such as ORB-SLAM3 [5] has made the state-estimation part of the problem widely reproducible, leaving depth sensing as the remaining cost driver on a small platform.

Dense two-frame stereo has a standard taxonomy and benchmark [1]. Within it, Semi-Global Matching (SGM) gives dense depth at a lower cost than fully global optimization [6]. Later work targets real-time execution: Gehrig and Rabe on multicore CPUs [7], Haller and Nedevschi [8] and Banz et al. [9] on GPUs, and Spangenberg et al. [10] through CPU parallelization. That line of work indentifies matching speed as a bottleneck. Learned matchers such as RAFT-Stereo [11] now lead the public benchmarks, at a computational cost that is harder to justify on a small drone. Our measurements point elsewhere: for consumer webcams in ordinary rooms, capture quality limits the result long before matching speed does.

III. Proposed Methodology

A. System Architecture

The proposed stereo-vision system has two cameras on a rigid mount with roughly parallel optical axes to fix their relative pose after calibration. The pipeline rectifies each pair, estimates disparity, converts it to metric depth, and back-projects the valid pixels into a colored point cloud. From that point cloud we read the visible size of a selected region and the clearance from the camera to the nearest visible surface in each direction. Fig. 1 shows the system pipeline and Fig. 2 shows the stereo geometry. All variables used are defined in Table I

TABLE I
TERMINOLOGY AND PARAMETER DEFINITIONS

Parameter	Symbol	Unit
Rectified focal lengths	f_x, f_y	px
Stereo-camera baseline	B	m
Horizontal image coordinates	x_L, x_R	px
Disparity	d	px
Image-pixel coordinates	u, v	px
Rectified principal point	c_x, c_y	px
Three-dimensional point coordinates	X, Y, Z	m
Lower and upper percentile limits	L, U	Dimensionless
Visible width, height, and depth	W, H, D	m
Visible bounding volume	V	m^3
Actual and estimated depth	Z_{actual}, $Z_{\text{estimated}}$	m
Absolute and percentage depth errors	$e_Z, e_{Z,\%}$	m, %
Disparity and depth uncertainty	$\delta d, \delta Z$	px, m
Disparity search bounds	$d_{\min}, d_{\max}$	px

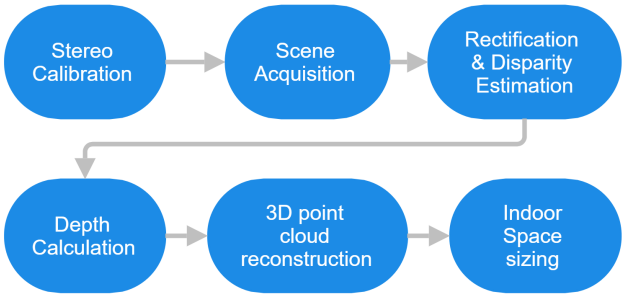


Fig. 1. System workflow pipeline for the proposed stereo-vision method.

B. Stereo Geometry

For parallel rectified cameras with focal length f_x and baseline B, let (X, Y, Z) be a scene point relative to the left camera. After subtracting the principal point, similar triangles can be expressed as:

$$\frac{x_L}{f_x} = \frac{X}{Z}, \tag{1}$$

$$\frac{x_R}{f_x} = \frac{X - B}{Z}. \tag{2}$$

The disparity between the projected coordinates is:

$$d = x_L - x_R \tag{3}$$

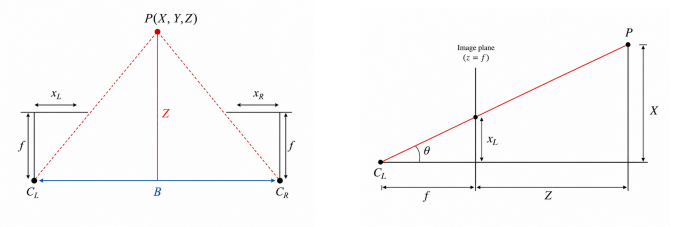
$$= \frac{f_x X}{Z} - \frac{f_x (X - B)}{Z} \tag{4}$$

$$= \frac{f_x B}{Z}. \tag{5}$$

Rearranging gives the forward depth:

$$Z = \frac{f_x B}{d}, \tag{6}$$

where f_x and d are measured in pixels and B is measured in meters. Larger disparities mean nearer points. The sign of d matters, because a scene point appears further to the right in the left image than in the right image, so d must be positive. A negative value of d means the views have been swapped, and one-directional matcher will then find nothing. Differentiating



(a) Stereo-camera geometry. (b) Similar triangles.

Fig. 2. Stereo geometry and similar-triangle depth derivation.

(6) gives the depth error caused by a disparity error δd,

$$\delta Z = \frac{Z^2}{f_x B}\, \delta d. \tag{7}$$

The error grows with the square of the range. For this rig ($f_x = 1089.4$ px, $B = 0.25$ m), a half-pixel matching error is 1.8 mm at 1 m, 29 mm at 4 m, and 118 mm at 8 m. Eq. (7) therefore sets the useful operating range of the rig. For pixel (u, v), back-projection into the left-camera frame gives:

$$X = \frac{(u - c_x)Z}{f_x}, \tag{8}$$

$$Y = \frac{(v - c_y)Z}{f_y}, \tag{9}$$

where (c_x, c_y) is the principal point and f_x, f_y are the rectified focal lengths. Applying these to each valid disparity reconstructs the visible scene.

C. Calibration and Rectification

We capture ChArUco pairs with the board at different positions and orientations. The board pairs a checkerboard with ArUco fiducial markers [12], so corner identity survives partial and oblique views. We then calibrate each camera for intrinsic and lens distortion following Zhang's method [13]. Stereo calibration finds the rotation and translation between the cameras, and rectification aligns matching points on the same image row.

A low reprojection error does not mean the calibration is usable, so we check three more things first and start with frame coverage. Because the distortion model is fitted only where the board was seen, we report the fraction of the image radius the board reached. Next we confirm which camera is used for the captured image. The operating-system camera indices can change between sessions, so we infer the left–right order from the sign of the board disparity rather than from the device index or file name. Last we measure the rectification residual, the median vertical difference between matched ChArUco corners. Block matching compares an $n \times n$ window along a single row, so a residual near $n/2$ leaves the

matcher comparing different surfaces while still returning a confident answer.

D. Disparity and Depth Estimation

We match densely with OpenCV StereoSGBM, based on Hirschmüller's semi-global method [6]. For each pixel it searches along the matching row in the other image and records the horizontal offset of the best-matching local pattern as the disparity. We derive the search range from the depth range of interest rather than fixing it in advance. Inverting (6):

$$d_{\min} = \frac{f_x B}{Z_{\max}}, \qquad d_{\max} = \frac{f_x B}{Z_{\min}}. \tag{10}$$

An over-wide range costs more than time. A pixel at column u can only be matched if $u > d_{\max}$, so the leftmost $d_{\max}$ columns of every row are discarded; at 1280 px, a 512-px range loses 40% of the frame before matching starts. A wide range also admits matches at physically impossible depths, and it sets the closest measurable surface: with $d_{\max} = 512$ px, nothing closer than 0.53 m can be measured. We therefore run a half-resolution pass to find which disparities the scene contains, then search only that range at full resolution. Several checks remove bad matches. A uniqueness check rejects matches whose best candidate is not clearly better than the rest, speckle filtering removes small inconsistent patches, and a left–right consistency check rejects pixels that disagree by more than one pixel. We also reject pixels with too little local contrast, pixels that are saturated to black, and pixels where the rectification map falls outside the source image. Depths outside the range in (10) are discarded. The colorized disparity map and generated depth map are shown in Fig. 5. Table IV shows the limits and parameters of the semi-global method

E. Point-Cloud Reconstruction and Space Sizing

We convert each pixel with a valid disparity into left-camera coordinates using (6), (8), and (9), and take its color from the left image. Invalid, nonfinite, and out-of-range points are removed. The rest pass through a local depth-consistency filter, are averaged onto a 2 cm voxel grid, and are stripped of points with too few neighbors. To measure the space, we use lower and upper percentiles of the coordinates instead of the extremes, which reduces the effect of outliers (bad points). The visible extents are:

$$W = X_U - X_L, \tag{11}$$

$$H = Y_U - Y_L, \tag{12}$$

$$D = Z_U - Z_L, \tag{13}$$

where W, H, and D are the visible width, height, and depth, and L and U are the chosen percentile limits. The bounding volume is:

$$V = WHD. \tag{14}$$

This is the extent of the visible points, not the enclosed room volume. When both floor and ceiling are visible, we take room height as the distance between the two fitted planes, which is more reliable than the extent of a single surface. We exclude surfaces seen within 25° of edge-on, because depth error stretches along their length. Modeling a full room would need extra viewpoints, pose estimation, and multi-frame fusion.

Algorithm 1 Stereo reconstruction and space sizing

Input: Stereo image pairs and calibration data
Output: Depth map, point cloud, and visible dimensions

Load camera matrices and rectification maps
for each left and right image pair **do**
  Find the true left–right order from the disparity sign
  Rectify both images; check the epipolar residual
  Convert to grayscale and enhance contrast
  Set $[d_{\min}, d_{\max}]$ from $[Z_{\min}, Z_{\max}]$ by (10)
  Compute disparity with StereoSGBM
  for each pixel in the disparity map **do**
    if disparity is positive, finite, and consistent **then**
      Compute $Z \leftarrow f_x B/d$
      if depth is inside the valid range **then**
        Back-project pixel to (X, Y, Z)
        Save 3D point and left-image color
      end if
    end if
  end for
  Remove invalid and isolated 3D points
  Compute lower and upper percentiles
  Compute W, H, D, and V
end for

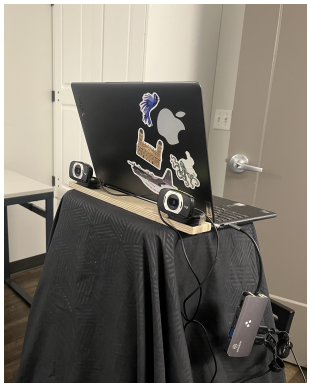


Fig. 3. Stereo test arrangement and ground-truth measurements.

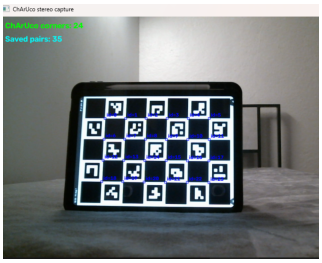
IV. Preliminary Results

A. Experimental Configuration

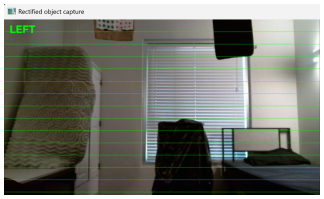
The prototype uses two Logitech C615 cameras with a measured 0.25 m baseline and captures 1280×720 stereo pairs. Testing used an Intel i9-10900K workstation with 32 GB of memory and an RTX 3080, and a Lenovo Yoga 9i with an i7-1360P, 16 GB of memory, and Iris Xe graphics. The OpenCV/NumPy implementation handles rectification, StereoSGBM matching, depth conversion, and point-cloud reconstruction. Fig. 3 shows the measurement setup.

B. Calibration and Rectification Results

Stereo calibration used 35 ChArUco image pairs. The left, right, and stereo RMS reprojection errors were 0.1025, 0.1022, and 0.2493 px. Rectification gave a 1089.4-px focal length and



(a) ChArUco calibration detection.



(b) Rectified pair with epipolar alignment lines.

Fig. 4. Calibration and rectification outputs.

TABLE II
STEREO CALIBRATION RESULTS

Parameter	Value
Accepted stereo pairs	35
Image resolution	1280 × 720 px
Left-camera RMS error	0.1025 px
Right-camera RMS error	0.1022 px
Stereo RMS error	0.2493 px
Median epipolar residual	0.49 px
Measured baseline	0.2500 m
Calibrated baseline	0.2453 m
Baseline difference	1.9%
Rectified focal length	1089.4 px
Rectified horizontal FOV	60.87°
Radial target coverage (weaker camera)	37%

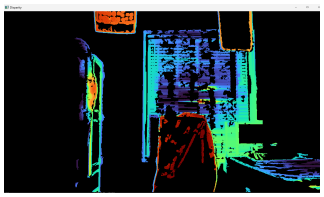
a 60.87° horizontal field of view, with a median epipolar residual of 0.49 px. Table II lists the results and Fig. 4 shows the calibration and alignment outputs. These numbers make sense because the board reached only 37% of the image radius in the weaker camera, so the distortion model is extrapolated over most of the frame despite the low reprojection error. Metric depth uses the measured baseline, with the calibrated baseline kept as a consistency check. These values are provisional until we recapture with better board coverage.

C. Validation Against a Known Reference

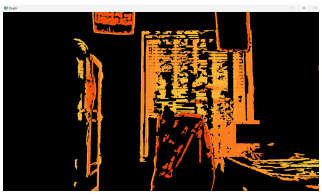
We rendered a stereo pair of a computer-generated room of known size, with surface texture, vignetting, and sensor noise, and ran it through the same code as the real captures. This isolates the error from the reconstruction process. Table III gives the results. All three dimensions land within 1%, and the median depth error is 16.9 mm with a 95th percentile of 34.0 mm. The pipeline recovers all five visible room surfaces as correctly oriented planes and returns depth for 89.0% of the frame. Setting the search range from (10) instead of using a fixed wide range raised depth completeness on this scene from 21.4% to 89.0%. The reconstruction is, therefore, not what limits the real experiments.

TABLE III
RECONSTRUCTION ACCURACY ON A GENERATED ROOM OF KNOWN SIZE

Quantity	Reference	Estimate	Error
Room width	4.000 m	4.007 m	0.18%
Room height	2.600 m	2.597 m	0.11%
Camera-to-wall distance	4.000 m	4.031 m	0.77%
Median depth error			16.9 mm
95th-percentile depth error			34.0 mm
Depth completeness			89.0%
Planar surfaces recovered			5 of 5



(a) Colorized disparity map.



(b) Filtered metric-depth map.

Fig. 5. Dense disparity and depth-estimation outputs.

D. Observed Failure Conditions

Real captures in a furnished bedroom returned depth for only 3.8% of the frame, against 89.0% for the generated scene. Two capture-side causes explain the gap. The first is scene texture. Only 27% of pixels had a local intensity standard deviation of at least three gray levels, against 99.8% for the generated room, and 9% of the frame was crushed to black. The largest apparently textured region was a set of window blinds, but their stripes run horizontally and provide no help in finding a match along a horizontal row. Painted walls, backlit windows, and dark fabric are common indoors and are close to the worst case for stereo. The second is rig movement after calibration. A pair captured after the rig was disturbed rectified with a nearly constant 39 px vertical offset, a rigid rotation of about 2°, while the same calibration rectified its own images to 0.10 px. A scene-based pitch and roll correction brought the residual only to 2.33 px, still large next to the matching window. The depths that survive are not just imprecise. Two patches on the same wall differed by 3.15 m, which is physically impossible. Interpolating into the untextured areas produced a dense, smooth-looking cloud from the same 3.8% of real measurements, with no gain in accuracy. We therefore leave interpolation off by default and count measured and interpolated pixels separately. Fig. 6 shows the point cloud and sizing output.

TABLE IV
STEREOSGBM PARAMETERS

Parameter	Value
Disparity search range	from (10), adaptive
Block size	9 px
Uniqueness ratio	12
Speckle window size	200 px
Speckle range	2
Left–right maximum difference	1 px
Minimum local intensity std. dev.	3 gray levels
Nominal depth range	0.60–10.00 m



(a) Colored three-dimensional point cloud.

(b) Estimated dimensions and bounding volume.

Fig. 6. Three-dimensional reconstruction and visible-space sizing.

E. Discussion

For consumer webcams in ordinary rooms, the capture of the images set the limits of the approach rather than the matching algorithm. Board coverage during calibration, keeping the rig still between calibration and capture, and having enough scene detail each matter far more than the algorithm error in Table III. On a real drone, a stereo rig needs a way to notice that its own calibration becomes obsolete, because a knocked rig returns a confident but wrong depth rather than an obvious failure. Continuous self-calibration, which tracks the camera parameters online instead of trusting a one-time calibration, is a more reliable alternative [14]. Because stereo cannot recover details that are not in the scene, projecting a pattern onto the scene would remove the texture problem at low cost and power, as Konolige et al. [15] demonstrated for indoor stereo.

V. CONCLUSIONS

We reported our work on a low-cost stereo system built from two Logitech C615 cameras, StereoSGBM, metric-depth conversion, and percentile-based point-cloud sizing. In a test environment with known dimensions the pipeline recovers width to within 0.18%, height to within 0.11%, and camera-to-wall distance to within 0.77%, which shows the reconstruction is not the limiting factor. Early real captures point instead to calibration coverage, rig movement after calibration, and scene texture, and we give a measurable check for each. The next step is a full accuracy test with controlled capture, comparing estimated depths and dimensions against tape-measured ground truth. We will plot errors against range and compare them with (7) across textured, textureless, repetitive, reflective, and dimly lit scenes, reporting depth completeness besides accuracy. Every capture will be screened first with the checks in Section III. The pipeline then moves onto drone hardware with synchronized cameras, where indoor flights can measure latency, power draw, and obstacle clearance. Adding pose estimation and multi-frame fusion would extend local sizing into a usable map.

REFERENCES

[1] D. Scharstein and R. Szeliski, "A taxonomy and evaluation of dense two-frame stereo correspondence algorithms," *Int. J. Comput. Vis.*, vol. 47, no. 1–3, pp. 7–42, Apr. 2002, doi: 10.1023/A:1014573219977.

[2] K. Mohta, K. Sun, S. Liu, M. Watterson, B. Pfrommer, J. Svacha, Y. Mulgaonkar, C. J. Taylor, and V. Kumar, "Experiments in fast, autonomous, GPS-denied quadrotor flight," in *Proc. IEEE Int. Conf. Robot. Autom. (ICRA)*, 2018, pp. 7832–7839, doi: 10.1109/ICRA.2018.8463214.

[3] B. Zhou, F. Gao, L. Wang, C. Liu, and S. Shen, "Robust and efficient quadrotor trajectory generation for fast autonomous flight," *IEEE Robot. Autom. Lett.*, vol. 4, no. 4, pp. 3529–3536, Oct. 2019, doi: 10.1109/LRA.2019.2927938.

[4] F. Kong, W. Xu, Y. Cai, and F. Zhang, "Avoiding dynamic small obstacles with onboard sensing and computation on aerial robots," *IEEE Robot. Autom. Lett.*, vol. 6, no. 4, pp. 7869–7876, Oct. 2021, doi: 10.1109/LRA.2021.3101877.

[5] C. Campos, R. Elvira, J. J. Gómez Rodríguez, J. M. M. Montiel, and J. D. Tardós, "ORB-SLAM3: An accurate open-source library for visual, visual-inertial, and multimap SLAM," *IEEE Trans. Robot.*, vol. 37, no. 6, pp. 1874–1890, Dec. 2021, doi: 10.1109/TRO.2021.3075644.

[6] H. Hirschmüller, "Stereo processing by semiglobal matching and mutual information," *IEEE Trans. Pattern Anal. Mach. Intell.*, vol. 30, no. 2, pp. 328–341, Feb. 2008, doi: 10.1109/TPAMI.2007.1166.

[7] S. K. Gehrig and C. Rabe, "Real-time semi-global matching on the CPU," in *Proc. IEEE Comput. Soc. Conf. Comput. Vis. Pattern Recognit. Workshops (CVPRW)*, 2010, pp. 85–92, doi: 10.1109/CVPRW.2010.5543767.

[8] I. Haller and S. Nedevschi, "GPU optimization of the SGM stereo algorithm," in *Proc. 6th IEEE Int. Conf. Intell. Comput. Commun. Process. (ICCP)*, 2010, pp. 197–202, doi: 10.1109/ICCP.2010.5606438.

[9] C. Banz, H. Blume, and P. Pirsch, "Real-time semi-global matching disparity estimation on the GPU," in *Proc. IEEE Int. Conf. Comput. Vis. Workshops (ICCVW)*, 2011, pp. 514–521, doi: 10.1109/ICCVW.2011.6130286.

[10] R. Spangenberg, T. Langner, S. Adfeldt, and R. Rojas, "Large scale semi-global matching on the CPU," in *Proc. IEEE Intell. Vehicles Symp.*, 2014, pp. 195–201, doi: 10.1109/IVS.2014.6856419.

[11] L. Lipson, Z. Teed, and J. Deng, "RAFT-Stereo: Multilevel recurrent field transforms for stereo matching," in *Proc. Int. Conf. 3D Vis. (3DV)*, 2021, pp. 218–227.

[12] S. Garrido-Jurado, R. Muñoz-Salinas, F. J. Madrid-Cuevas, and M. J. Marín-Jiménez, "Automatic generation and detection of highly reliable fiducial markers under occlusion," *Pattern Recognit.*, vol. 47, no. 6, pp. 2280–2292, Jun. 2014, doi: 10.1016/j.patcog.2014.01.005.

[13] Z. Zhang, "A flexible new technique for camera calibration," *IEEE Trans. Pattern Anal. Mach. Intell.*, vol. 22, no. 11, pp. 1330–1334, Nov. 2000.

[14] T. Dang, C. Hoffmann, and C. Stiller, "Continuous stereo self-calibration by camera parameter tracking," *IEEE Trans. Image Process.*, vol. 18, no. 7, pp. 1536–1550, Jul. 2009, doi: 10.1109/TIP.2009.2017824.

[15] K. Konolige, "Projected texture stereo," in *Proc. IEEE Int. Conf. Robot. Autom. (ICRA)*, 2010, pp. 148–155.

[16] X. Zhao, W. Ding, Y. An, Y. Du, T. Yu, M. Li, M. Tang, and J. Wang, "Fast Segment Anything," arXiv:2306.12156, 2023.